9-10-2026

N Reynas



DIEGO PACHECO PÉREZ
IGNACIO RAMOS VILLA

1. Descripción del problema

El problema de las 8 reinas consiste en colocar 8 reinas en un tablero de ajedrez de 8×8 de tal forma que ninguna reina pueda atacar a otra.

En ajedrez, una reina puede moverse:

- Horizontalmente
- Verticalmente
- Diagonalmente

Por lo tanto, una solución válida debe evitar:

- Dos reinas en la misma fila
- Dos reinas en la misma columna
- Dos reinas en la misma diagonal

Es un problema de optimización combinatoria y puede resolverse mediante técnicas de búsqueda como la búsqueda Tabú, la cual evita ciclos y óptimos locales.

2. Representación de P y S

Problema (P)

Encontrar una configuración de 8 reinas en un tablero 8×8 sin conflictos.

Solución (S)

Se representa mediante un arreglo:

$$S = [4, 2, 7, 3, 6, 8, 5, 1]$$ Donde:

- Índice = columna
- Valor = fila Ejemplo:

- $S[0] = 4$ → Reina en columna 1, fila 4

Esta representación permite que no haya conflictos por columnas y no haya conflictos por filas (porque es permutación) solo se evalúan conflictos diagonales.

3. Propuesta de algoritmo (Seudocódigo)

Generar una solución inicial aleatoria S
Evaluar su costo (conflictos)

```
Mejor ← S
ListaTabu ← vacía

Mientras no se cumpla condición de paro:

   Generar vecinos de S
   Evaluar cada vecino
   Ordenar vecinos por menor costo

   Para cada vecino:
      Si no está en lista tabú:
         S ← vecino
         Agregar vecino a lista tabú

         Si mejora:
            Mejor ← vecino

         Romper ciclo

   Si lista tabú supera tamaño:
eliminar el más antiguo

   Si costo es 0:
terminar

Retornar mejor solución encontrada
```

4. Implementación en Python

Está anexada en el repositorio de Github

5. Métricas implementadas

El algoritmo incluye las siguientes métricas:

- **Tiempo de ejecución**

```
inicio = time.time() fin
= time.time()
```

- **Cantidad de movimientos**

```
movimientos += 1
```

- **Número de conflictos**

mejor_costo

6. Ejemplo de salida

Mejor solucion: [4, 2, 7, 3, 6, 8, 5, 1]
Conflictos: 0
Movimientos: 120
Tiempo: 0.01 segundos

7. Cumplimiento de reglas

Estado inicial aleatorio:

random.shuffle(sol)

Métricas de rendimiento:

- Tiempo
- Movimientos
- Conflictos Preparado para pruebas en clase:

- Parámetros ajustables:

tabu(iteraciones=5000, limitetabu=50, n_vecinos=30)